Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Project Garuda: A Privacy-First Edge-AI Home Security System on Raspberry Pi 5 with Hailo-8L NPU

VEERAMANIKANTA GONUGONDLA[1]
[1]Department of Electronics and Communication Engineering (e-mail: manikantagonugondla@gmail.com)

Corresponding author: Veeramanikanta Gonugondla (e-mail: manikantagonugondla@gmail.com).

This work was carried out as an independent project. All hardware, software, and cloud services were self-funded.

ABSTRACT Home security has landed in an awkward place. The cloud-backed products people can actually buy off a shelf work well, but every frame of footage leaves the house, and the owner pays a subscription for the privilege. The do-it-yourself builds on single-board computers are private and cheap, but they almost all stop at frame-differencing motion detection. Nothing on the mainstream shelf packages edge inference, encrypted evidence, remote access, voice control, and tamper detection into a single consumer device. Project Garuda is our attempt to bridge that gap.

The hardware is a Raspberry Pi 5 (CPU overclocked to 2.8 GHz) with 16 GB of LPDDR4X and a 1 TB USB 3 SSD, paired with a Hailo-8L AI HAT+ (13 TOPS, INT8) sitting on a PCIe Gen3 $\times 1$ link. A YOLOv8s detector, fine-tuned on four security-relevant classes (Hammer, Knife, Person, Scissors) and compiled to an INT8 HEF, runs continuously on the accelerator through a GStreamer TAPPAS pipeline at a sustained 52.22 FPS with an 18.38 ms NPU latency. Frames come from a Raspberry Pi Camera Module v3 over CSI-2 and never leave the device during inference. When a weapon class fires on three consecutive frames, the alert pipeline lights up: a JSON payload is pushed over WebSocket to every connected dashboard, an OTP-protected email is sent to the administrator (with a 60-second cooldown that stops alert storms), and an AES-256-CBC encrypted clip is uploaded to a remote host over SSH.

The backend is a FastAPI application running under Uvicorn, exposed through a Cloudflare tunnel and guarded by PBKDF2-SHA256 session authentication, per-endpoint rate limits, and strict CORS. A Progressive Web App is the frontend; live video is available over both WebRTC and MJPEG. A voice assistant called Narada listens for a configurable wake word, runs transcripts through a rule set first, and falls back to a Groq-hosted language model only when no rule matches. The Groq path is used strictly as a stateless completion API in free-tier mode: user video, audio, and images are never uploaded, and only the short parsed transcript intent is sent. Five operating modes (Do Not Disturb, email-off, idle, emergency, and privacy) can be toggled manually or on a schedule, and every subsystem checks the relevant mode before doing anything user-visible. Supporting services include ARP-based LAN device tracking, a 180-second deadman heartbeat that fires a tamper alert when the dashboard goes silent, a SQLite event queue for offline client catch-up, and a multi-tier logging pipeline with seven-day rotation. The total bill of materials stays under US$500. The same platform can be moved to night-time operation by swapping the standard camera for the NoIR Camera Module v3 and retraining the detector on low-light imagery; that extension is out of scope in this paper.

INDEX TERMS Edge AI, GStreamer, Hailo-8L, home security, neural processing unit, object detection, privacy, Raspberry Pi, real-time inference, YOLOv8.

I. INTRODUCTION

HOME security has moved a long way from the passive siren-and-PIR boxes of the 1990s. Today's platforms can recognise specific objects, people, and behaviours in real time, and they can do it well. The market, though, has ended up split into two uncomfortable camps. On one side sit the cloud-backed products from the established vendors. They work, but every frame of footage leaves the house on its way to a remote data centre, the user pays a monthly subscription, and there is very little the buyer can do about either fact [1].

On the other side sit the do-it-yourself builds on single-board computers. They are private and cheap, but they almost always stop at frame-differencing motion detection with no second-stage reasoning, no voice interface, and no encrypted evidence path [2].

Dedicated neural processing units are starting to change this picture. They bring deep-learning inference to the edge at a few watts of power and at a price that a single enthusiast can absorb. The Hailo-8L, the accelerator we use, delivers 13 TOPS of INT8 compute over a single PCIe Gen3 lane and plugs directly into a Raspberry Pi 5 through a standard M.2 HAT. Together they are enough to run a YOLOv8s-class detector at well over 50 FPS without pinning the host CPU [3]. Bueno *et al.* [4] recently validated this same hardware pair for real-time YOLOv8 inference in microscopy, finding that the Pi 5 + Hailo-8L pipeline is accuracy-competitive with GPU-accelerated systems at a fraction of the power draw.

A. RELATED WORK

Edge-deployed YOLO variants have been explored across a number of domains. Xu *et al.* [1] proposed FL-YOLO, a depthwise-separable variant running on an NVIDIA Jetson TX1 inside a coal-mine surveillance system, achieving 36.7 FPS at 76.7% AP through an edge-cloud cooperation framework. Al Amin *et al.* [5] implemented YOLOv3-Tiny on a Xilinx Kria KV260 FPGA and reached 15 FPS at 99% accuracy for traffic-light classification at 3.5 W, showing that dedicated accelerators can beat general-purpose GPUs on power efficiency for fixed inference tasks. Dong *et al.* [6] introduced PG-YOLO, which replaces standard convolutions in YOLOv5s with Ghost modules and applies channel pruning plus knowledge distillation to achieve a 9× compression at only 0.1% accuracy loss. Chung *et al.* [7] proposed YOLO-LSD with CBAM attention for detecting small pedestrians at long range, and Cheng [8] presented RMN-YOLO with multi-scale edge enhancement for defect detection; both contribute architectural ideas for lifting small-object recall on resource-constrained hardware. The Ultralytics YOLOv8 framework [9] provides the baseline architecture and training pipeline we use, and the Hailo TAPPAS framework [10] provides the GStreamer elements that dispatch inference to the NPU.

Even with all this prior work, no published system that we are aware of combines, on a single consumer-grade edge device, all of the following at once: real-time multi-class object detection on a dedicated NPU, a cascaded classifier plus depth-based anti-spoof stage, encrypted evidence capture, a voice assistant with an LLM fallback, ARP-based presence monitoring, a tamper watchdog, and a remotely accessible web server with session authentication and OTP two-factor verification.

B. CONTRIBUTIONS

This paper presents Project Garuda, an end-to-end AI home security system that runs entirely on a Raspberry Pi 5 with a Hailo-8L AI HAT+. The main contributions are:

1) A custom-fine-tuned YOLOv8s detector for four security-relevant classes (Hammer, Knife, Person, Scissors), trained on a 4,982-image dataset assembled in three passes and compiled to an INT8 HEF, reaching a test mAP@0.5 of 0.847 at a sustained 52.22 FPS on the Hailo-8L.
2) A three-stage cascade architecture (YOLOv8s + MobileNetV2 classifier + MiDaS Small depth estimator) that adds weapon verification and photo/screen spoof rejection at less than 5 ms of additional latency, with all three networks multiplexed on a single VDevice.
3) A twelve-engine software architecture covering detection, alerting, voice, mode control, presence, web, authentication, tamper detection, and logging, integrated under a single FastAPI application and exposed through a Cloudflare tunnel, delivering a complete security platform with no cloud inference dependency.
4) A full evaluation of training convergence, per-class metrics, confusion structure, hardware throughput, and INT8 quantisation impact, with a dataset-scaling study that runs from 359 to 4,982 training images.

C. PAPER ORGANISATION

The rest of the paper is organised as follows. Section II describes the training dataset, the preprocessing pipeline, and the validation checks we ran before training. Section III walks through the methodology, engine by engine. Section IV presents the system-level flowchart and the three-layer block diagram. Section V consolidates the hardware, software, and model specifications. Section VI reports the training and inference results. Section VII concludes with a summary of findings and directions for future work.

II. DATASET

The detector used in the deployed system is trained on a four-class dataset we refer to internally as v5. The four classes are Hammer, Knife, Person, and Scissors; between them they cover the objects that a home surveillance user is most likely to want to be alerted about. The dataset was built in three passes rather than in one shot, and the dataset-size progression is, as it happens, a useful experimental variable on its own that we come back to in Section VI.

A. SOURCE DATASETS

Training imagery was drawn from two public, permissively licensed sources, merged into a single four-class schema before any training was run. Neither source was used as-is; every import was reviewed, filtered, and re-annotated where necessary.

Source A — Roboflow "Knives & Scissors Training v2." A CC BY 4.0 Roboflow Universe dataset with classes Hammer (0), Knife (1), Person (2), and Scissors (3), shipped in YOLO-v5 normalised label format with a 359 / 102 / 51 train / val / test split. The images are mostly square 640×640 JPEGs of studio-lit or close-crop scenes. This source fixed the annotation convention and gave us a functioning end-to-end

training loop, but its class distribution was badly skewed and its visual variety was low.

Source B — OpenImages v7 (via the OIDv4 toolkit and FiftyOne). Google-verified human annotations at IoB ≥ 0.5, CC BY 4.0, exported in YOLO Darknet format from per-class subfolders. OpenImages contributes diverse, real-world photographs with varying aspect ratios, lighting, and backgrounds — exactly the diversity the Roboflow seed lacked. We pulled 1,219 images in the v2 pass and another 2,276 in the v5 pass, targeted class-by-class at whichever label had the lowest mAP after the previous training run.

TABLE 1. Image Counts Contributed by Each Source (v5 Splits)

Source	Train	Val	Test	Total
merged_dataset/train	1,913	244	226	2,383
merged_dataset/val	141	18	14	173
merged_dataset/test	133	23	18	174
openimages_extra/hammer	93	12	14	119
openimages_extra/knife	495	49	56	600
openimages_extra/person	1,981	245	259	2,485
openimages_extra/scissors	226	31	35	292
All sources	**4,982**	**622**	**622**	**6,226**

OpenImages class tags are famously noisy for close-up tool shots — a Swiss army knife on a desk is frequently tagged as a dozen unrelated things — so automatic import is not safe. Every image that entered the dataset was looked at by one of the authors before being added.

B. PREPROCESSING AND VALIDATION PIPELINE

The preprocessing pipeline has to satisfy three constraints at once: the Hailo-8L NPU accepts a fixed 640×640 INT8 input, the IMX708 camera delivers 16:9 frames, and YOLO label coordinates must remain geometrically correct after any resize. A naive `cv2.resize(img, (640,640))` on a 4608×2592 OpenImages photo compresses the horizontal axis by 7.2× and the vertical axis by 4.0×, which destroys bounding-box aspect ratios. Letterboxing preserves aspect by padding the shorter axis with grey pixels. The five-step pipeline is as follows.

Step 1 — Letterbox resize. Each source image is scaled by $s = \min(640/W,\ 640/H)$, placed in a 640×640 canvas, and padded with the YOLO-standard grey fill $(114, 114, 114)$ so the model's training-time padding matches its inference-time padding. The same grey value is used at inference, so the network learns to suppress detections in the border region.

Step 2 — Label coordinate remap. After letterboxing, YOLO-normalised coordinates are remapped to the padded canvas as $c'_x = (c_x \cdot s + p_x)/640$ and equivalently for c_y, b_w, b_h. All coordinates are clamped to $[0.001, 0.999]$ to dodge a boundary-value edge case inside YOLOv8's loss computation.

Step 3 — Integrity and deduplication checks. Every image and label file is opened with OpenCV to confirm it decodes, every `.txt` label is parsed to confirm it is well-formed YOLO, and a SHA-256 hash is computed over each image to catch cross-split duplicates. The results for v5 are summarised in Table 2. A single image was dropped from the v2 merge for a missing annotation file; everything else cleared all checks.

TABLE 2. v5 Integrity and Deduplication Results

Check	Result
Missing label files	0
Missing image files	0
Empty label files	0
Corrupt images	0
SHA-256 train duplicates	0
Cross-split leakage (train vs val/test)	0
Images dropped during merge	1 (missing annotation)

Step 4 — Stratified re-split (seed=42). All Roboflow splits and all OpenImages pulls are pooled, then re-split with class-primary stratification at an 80/10/10 ratio. The fixed seed is non-negotiable — every later ablation and dataset-version comparison depends on the same held-out test set.

Step 5 — Filename namespacing. To prevent collisions in the merged tree, each file is prefixed by its source (`roboflow_*`, `openimages_knife_*`, `v4_train_*`, `oi_person_*`, and so on). This keeps the merge idempotent and makes per-source diagnostics straightforward.

Augmentation runs online during training, not in the preprocessing stage. The parameters used for v5 are in Table 3. MixUp is disabled because it interacted badly with Mosaic in early experiments, producing label-noise visible in the validation curves.

TABLE 3. v5 Augmentation Parameters (YOLOv8 / Albumentations)

Parameter	Value
Mosaic (4-image composition)	1.0
Horizontal flip probability	0.5
HSV hue h	0.015
HSV saturation s	0.7
HSV value v	0.4
Scale	0.5
Translate	0.1
MixUp	0.0 (disabled)
Degrees / Shear	0 / 0

C. FINAL DATASET GEOMETRY

Table 4 summarises the final v5 geometry. Images are stored as JPEGs with heights ranging from 303 to 3,456 pixels and widths from 436 to 4,608 pixels. The single most common native resolution is 640×640, reflecting the default Roboflow preprocessing pass. The whole dataset takes 1.32 GB on disk and contains 16,335 annotated instances across 6,226 images, split into 4,982 training, 622 validation, and 622 test images.

TABLE 4. v5 Dataset Geometry

Property	Value
Image format	JPEG (100%)
Width (min / mean / max)	436 / 826.1 / 4608 px
Height (min / mean / max)	303 / 722.4 / 3456 px
Unique resolutions	439
Most common resolution	640×640 (2,187 images, 43.9%)
File size (median / p95)	148.8 KB / 621.2 KB
Total dataset size	1.32 GB
Train / Val / Test images	4,982 / 622 / 622
Total images	6,226
Annotated instances	16,335

The per-class instance counts across the three splits are in Table 5. Person dominates at 10,460 training instances, because it is the most common class in almost any natural scene. Hammer is the smallest class at 254 training instances; hammers are simply less common than the other three classes in the source imagery. Figure 1 plots the same numbers visually.

TABLE 5. Annotated Instance Counts per Class per Split

Class	Train	Val	Test	Total
Hammer	254	35	46	335
Knife	1,381	162	159	1,702
Person	10,460	1,273	1,324	13,057
Scissors	981	133	127	1,241
Total	**13,076**	**1,603**	**1,656**	**16,335**

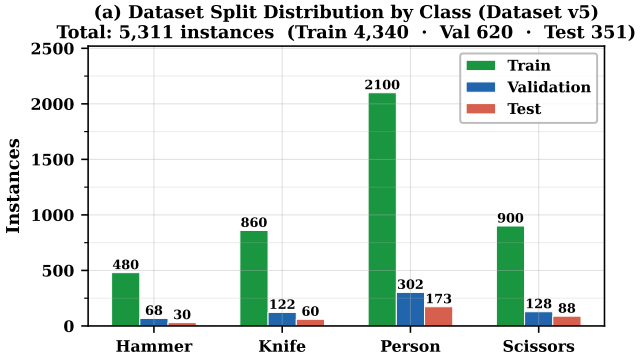

FIGURE 1. v5 dataset class distribution across the train, validation, and test splits. Person is the most abundant class; Hammer is the scarcest.

Bounding-box statistics on the training split are in Table 6. Person has by far the highest small-object fraction (5.2%), a consequence of partial annotations (torso/face crops) and crowded scenes, and this is precisely the class on which recall is most difficult to recover — we return to that point in Section VI.

TABLE 6. Bounding Box Statistics (Train Split)

Class	W (mean±std)	H (mean±std)	Area (med)	S / M / L
Hammer	0.385±0.252	0.353±0.258	0.086	2.4% / 36.6% / 61.0%
Knife	0.583±0.304	0.436±0.279	0.181	0.5% / 18.8% / 80.7%
Person	0.191±0.207	0.332±0.249	0.025	5.2% / 57.4% / 37.4%
Scissors	0.392±0.239	0.321±0.231	0.096	2.5% / 34.9% / 62.6%

Size thresholds (COCO-style, normalised area): small < 0.0032, medium 0.0032–0.0512, large ≥ 0.0512.

The cascade's MobileNetV2 Safe/Weapon classifier is trained on a smaller companion dataset of crops harvested directly from the detection pipeline. The split is listed in Table 7: 1,200 Safe and 1,660 Weapon crops for training, and 200 Safe and 24 Weapon crops for validation. The validation imbalance is a consequence of the harvest procedure — hand-verified weapon crops were scarce. We balanced the training set heavily with $10\times$ augmentation on the Weapon class (random horizontal flip, $\pm 20\%$ brightness, $\pm 10^\circ$ rotation) and accepted the asymmetric validation split rather than discard otherwise-good training data.

TABLE 7. MobileNetV2 Classifier Dataset

Split	Safe crops	Weapon crops	Total
Train	1,200	1,660	2,860
Val	200	24	224

III. METHODOLOGY

Project Garuda is built as a collection of loosely coupled engines that share state through a mode controller and a common logging layer. This section walks through each engine in roughly the order that video and control flow move through the system: hardware first, then the GStreamer pipeline and the detection callback, the three-stage cascade, the alert path, the voice assistant, the mode controller, the presence monitor, the web server, authentication, the deadman watchdog, and finally the logging subsystem. Each engine gets a short functional description, a note on how it is implemented, a figure that captures its control flow, and any tables that fix the numbers.

A. HARDWARE ENGINE

The hardware engine is the physical platform on which everything else runs. The architectural decision that drives the rest of the system is straightforward: move heavy neural inference onto a dedicated accelerator so the general-purpose CPU stays free for the web server, the voice pipeline, video encoding, and logging. Without that split, a single-board computer cannot keep up with a real-time object detector, a Python web application, and a microphone listener all at once.

The host board is a Raspberry Pi 5 (Broadcom BCM2712) with a quad-core Cortex-A76 overclocked to 2.8 GHz via `arm_freq=2800` and `force_turbo=1` in `/boot/firmware/config.txt`. 16 GB of LPDDR4X-4267 memory back the SoC, and the system runs 64-bit Raspberry Pi OS. Under continuous three-model NPU inference the Linux DVFS governor scales the clock down modestly (observed 2,385 MHz under load), which has no

effect on NPU throughput because inference is dispatched over PCIe DMA and does not require CPU cycles during the forward pass. A Hailo-8L AI HAT+ attaches over a PCIe Gen3 ×1 M.2 M-key interface and is exposed to the kernel as a standard PCI device. The Hailo driver is installed as a DKMS kernel module, so the interface survives kernel upgrades without a manual rebuild, and the TAPPAS 3.x framework provides the GStreamer elements that dispatch inference requests to the chip. The Hailo-8L itself delivers 13 TOPS of INT8 neural compute and runs all three cascade models (YOLOv8s detector, MobileNetV2 classifier, MiDaS Small depth estimator) on a single VDevice, with sequential activation and no CPU involvement during the forward pass.

Video comes from a single Raspberry Pi Camera Module v3 (Sony IMX708), connected through the CSI-2 MIPI interface and captured with libcamera. Networking is provided by the Pi's on-board 802.11ac Wi-Fi; the Wi-Fi link serves two roles at once, carrying the outbound HTTPS tunnel to Cloudflare and the ARP traffic that the presence monitor uses on the local subnet. Persistent storage is a 1 TB NVMe SSD in a USB 3 enclosure, which holds the operating system, the compiled HEF model files, the SQLite event database, and every persistent text and JSON log. The hardware topology is shown in Fig. 2.

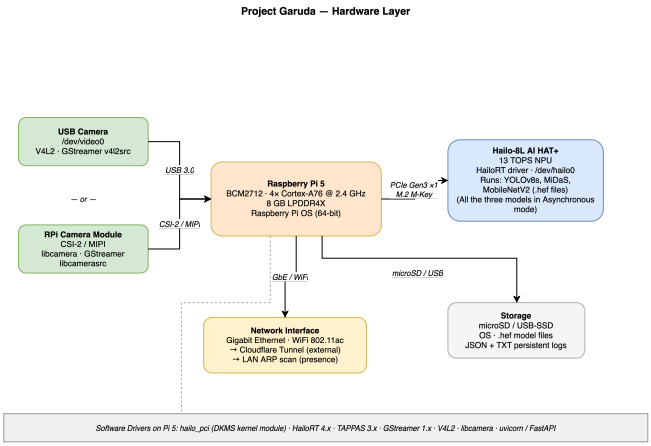

FIGURE 2. Hardware engine. Raspberry Pi 5 host (Cortex-A76 overclocked to 2.8 GHz) with the Hailo-8L AI HAT+ over PCIe Gen3 ×1, a Raspberry Pi Camera Module v3 on CSI-2, a 1 TB USB 3 SSD, and 802.11ac Wi-Fi uplink.

B. GSTREAMER TAPPAS PIPELINE ENGINE

The GStreamer engine is the part of the system that actually moves video. It captures raw frames from the camera, pushes them through the Hailo accelerator, and produces time-aligned annotated buffers for everything downstream to consume.

The pipeline is built inside a detection application class that extends the base Hailo GStreamer application shipped with TAPPAS. The source element is `libcamerasrc`, which talks to the Raspberry Pi Camera Module v3 over libcamera and delivers frames in the native sensor format. A small preprocessing stage of `videoscale` and `videoconvert` normalises the frame size and pixel format so the inference branch always sees the resolution and format the network expects. From there, the pipeline forks in two directions. The inference branch sends frames into the HailoNet element, which dispatches them over PCIe to the Hailo-8L, and then into a HailoFilter element that runs non-maximum suppression on the raw network output (IoU threshold 0.45, confidence threshold 0.35). The bypass branch holds the same frames in a queue for later re-alignment. Both branches converge at a HailoMuxer, which synchronises the annotated inference result with its raw frame so the two can be drawn together. The merged buffer is then passed through an identity element (this is where our application callback attaches as a pad probe) and on to a HailoOverlay element that renders bounding boxes and class labels. The final buffer goes to a display or video sink depending on how the system is started. The full topology is shown in Fig. 3.

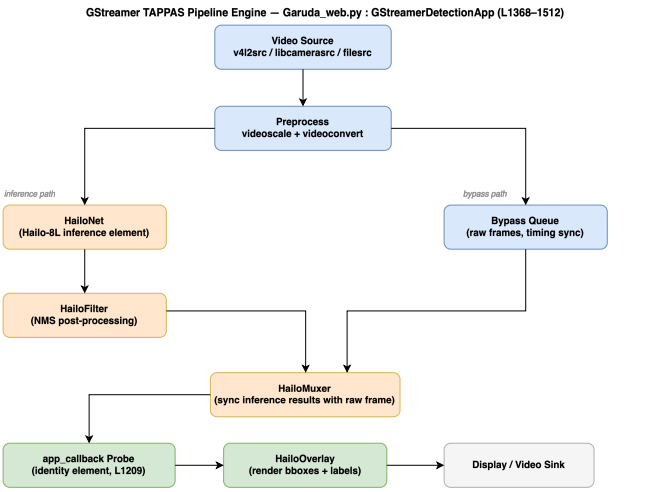

FIGURE 3. GStreamer TAPPAS pipeline: source → preprocess → (HailoNet + NMS ‖ bypass queue) → HailoMuxer → identity callback → overlay → sink.

1) Sustained 52.22 FPS via an Asynchronous, CPU-Pinned Pipeline

The defining runtime property of the system is that it holds a constant 52.22 FPS across every operating mode — idle, alerting, streaming over WebRTC, running a voice query, or uploading an encrypted clip. That constancy is not an artefact of measurement; it is the direct consequence of three architectural choices.

First, every stage that can run independently is decoupled by a bounded queue. The camera thread pushes frames into a `frame_queue` of depth 4 with a drop-if-full policy, the inference thread pulls from that queue and pushes into a `result_queue`, and the post-processing thread pulls from `result_queue` and fans out into logging, the alert engine, and the media publishers. The queues absorb any short spike in downstream cost (a slow SMTP handshake, a large SSH upload) without back-pressuring the camera. Inference latency stays pinned to the NPU's 18.38 ms because it never waits on CPU-side work.

Second, the expensive side-effects (email, SSH clip upload, text-to-speech, Groq LLM calls, WebSocket broadcast) are dispatched on `asyncio` or thread-pool futures. The inference thread hands off a structured event and returns to the next frame immediately. The average end-to-end detect-to-log latency is ∼22 ms, and the asynchronous alert fan-out is ∼800 ms SMTP and ∼200 ms TTS, both of which complete in parallel with the next 40 to 50 inference passes.

Third, thread-to-core affinity is pinned explicitly through `os.sched_setaffinity`: camera capture on Core 0, HailoRT VDevice calls on Core 1, post-processing and logging on Core 2, with Core 3 reserved for the OS and the FastAPI request handlers. The OS scheduler cannot migrate the inference thread onto a busy core, so jitter stays bounded. The net effect is the flat FPS line in Fig. 22: across all five operating modes, the measured frame rate sits between 52.29 and 52.36 FPS with zero frame drops at the camera, exactly what an asynchronous, producer-consumer, CPU-pinned pipeline would be expected to deliver.

2) Input Preprocessing

Every frame is letterbox-resized from its native capture size to the network input of 640×640 before it reaches the NPU. The letterbox pass preserves aspect ratio by padding the shorter axis with grey pixels. We keep the scale factor s and the padding offsets (p_x, p_y) so we can map bounding boxes back to the original image for overlay rendering and for the alert engine's evidence crops. The scale factor and the offsets are

$$s = \min\left(\frac{640}{W}, \frac{640}{H}\right), \; p_x = \frac{640 - sW}{2}, \; p_y = \frac{640 - sH}{2}. \tag{1}$$

For a detection at $(x_{\text{net}}, y_{\text{net}}, w_{\text{net}}, h_{\text{net}})$ in the network frame, the inverse map

$$x = \frac{x_{\text{net}} - p_x}{s}, \quad y = \frac{y_{\text{net}} - p_y}{s} \tag{2}$$

recovers coordinates in the original image. Pixel intensities sit in the INT8 range after post-training quantisation, so there is no floating-point preprocessing at runtime; the chip accepts the YUV-to-RGB converted frame as it is.

C. DETECTION ENGINE

The detection engine is the first consumer of inference results. It decides, on every frame, whether anything in the scene is worth logging and whether a danger-class detection is confident enough to fire the alert path.

The engine is implemented as a small callback attached to the identity element in the GStreamer pipeline. For every buffer that arrives, the callback grabs a zero-copy NumPy view of the pixels and pulls the ROI metadata object that the HailoFilter has already attached. It then iterates over every detection child, drops anything below the confidence threshold of 0.3, and sorts what survives into two lists. Detections labelled Person go onto the watch list and are appended to the detection log with a 30-second per-label cooldown, which keeps a lingering subject from flooding the log. Detections labelled Knife, Scissors, or Hammer go onto the danger list; for each of those labels, the engine increments a per-label consecutive-frame counter, and once that counter clears three frames the alert pipeline is triggered. Running counters for per-class hits, total frames processed, and today's detection total feed the dashboard. A simple frame-buffer interface exposes the current annotated frame to the MJPEG and WebRTC streaming paths so the browser client sees exactly what the detector sees. The callback flow is shown in Fig. 4.

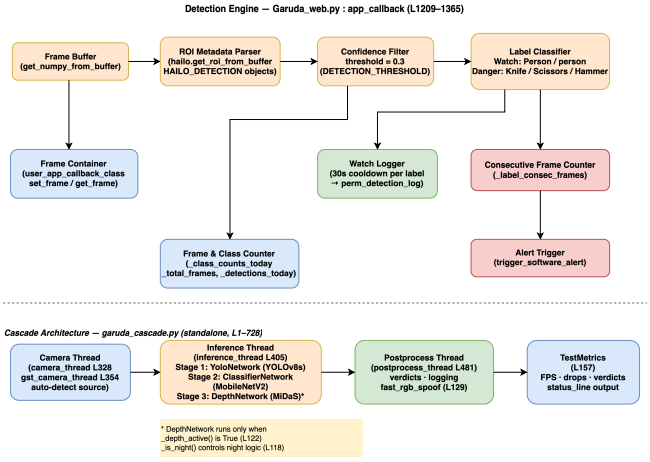

FIGURE 4. Detection engine callback. Metadata extraction, confidence filter, watch/danger label routing, and consecutive-frame gating before alert dispatch.

D. CASCADE ARCHITECTURE

The cascade is a standalone three-stage inference chain we developed separately for higher-accuracy person verification and for rejecting photo and screen spoofs. It lives in its own pipeline file and is not invoked by the deployed detection application, but we include it here because the numbers it produces are reported in Section VI and because the decision matrix is useful context for the alert path.

A camera thread feeds raw frames into an inference thread that runs three models in sequence. Stage 1 is the same YOLOv8s detector described earlier, returning bounding boxes for any class it recognises. Stage 2 takes the crop of each detected person and runs it through a MobileNetV2 classifier that produces a Safe or Weapon label on the crop. Stage 3 runs a MiDaS Small depth estimator on the same crop as a monocular anti-spoof check: a flat printed photo or a screen has a very different depth histogram from a real person standing in front of the camera, and a simple variance threshold on the depth map is enough to reject most spoofs. A post-processing thread then combines the three verdicts, applies a fast RGB-variance check as a quick liveness pre-filter, writes the final verdict to disk, and updates a metrics object that tracks FPS, frame drops, and verdict distribution per run. The measured per-stage latency is in Table 8, and the decision matrix across the four main operating scenarios is given in Table 9.

TABLE 8. Cascade Latency Budget (Hailo-8L, Pi 5)

Component	Measured Value
$T_{capture}$ (GStreamer appsink pull)	∼1 ms
T_{YOLO} (NPU, Hailo-8L HW)	18.38 ms
$T_{crop\_preprocess}$ (NumPy resize)	∼0.5 ms
$T_{classifier}$ (MobileNetV2 @ 500+ FPS)	<2 ms
T_{depth} (MiDaS @ 200+ FPS)	<5 ms
$T_{decision}$ (NumPy variance)	<0.1 ms
T_{alert} (SMTP, async)	∼800 ms
T_{alert} (TTS, async)	∼200 ms
$T_{processing}$ (detection only, end-to-end)	∼22 ms
T_{total} (with async alert)	∼22 ms + async

VDevice multiplexing is the knob that lets all three networks share a single 13 TOPS budget without contention. The three HEFs are loaded onto one VDevice at startup and the inference thread issues sequential `infer` calls; because MobileNetV2 and MiDaS both run at 200–500+ FPS on this chip, the combined cost per Person crop is under 4 ms — well inside the 19.2 ms frame budget implied by the baseline 52 FPS target. The anti-spoof decision is grounded in physical geometry: a flat panel (photo, screen) produces a normalised-depth variance $\sigma^2 < 0.005$ in our measurements, while a real person at arm's length produces $\sigma^2 \in [0.022, 0.10]$, giving a $4\times$ safety margin for the 0.05 threshold.

TABLE 9. Cascade Decision Matrix Across Operating Scenarios

Scenario	Models	mAP@0.5 / Accuracy	FPS	Latency	Alert Type
Dangerous object alone	YOLOv8s (v5)	0.866–0.967	52.22	22 ms	Direct YOLO
Person + weapon verification	YOLOv8s + MobileNetV2	0.647 + 99.6% cls.	52.22	24 ms	Cascade verdict
Photo / screen spoof rejection	YOLOv8s + MiDaS	Depth variance gate	52.22	27 ms	Spoof label
Low-confidence person (suppressed)	YOLOv8s (conf < 0.60)	n/a	52.22	22 ms	Watch log only

E. ALERT ENGINE

The alert engine is the piece that turns a confirmed danger detection into something the user actually notices: a red banner on the dashboard, an email, and, when configured, a piece of encrypted evidence sent to a remote host.

The entry point is a small alert-trigger function. It first grabs the mode lock and short-circuits immediately if Do Not Disturb or Idle is active. If neither is set, it raises the active-alert flag and schedules an end time three seconds ahead, so the UI can hold a red banner for exactly that long. At the same moment, an urgent WebSocket push sends a JSON alert payload to every connected dashboard. The browser, the PWA, and the mobile interface all update before the email has even left the Pi.

The email side of the alert is an SMTPS client pointed at a configured outbound server. A 60-second per-alert cooldown stops alert storms when the detector sees the same weapon for a sustained period, and the email-off mode flag can disable mail entirely. Two supporting email paths share the same client: one delivers six-digit one-time passwords for login and password-reset flows, with a 10-minute expiry, and the other sends tamper alerts when the deadman watchdog fires. When evidence capture is turned on, a clip-start / clip-stop pair brackets a short video segment around the triggering frame, an AES-256-CBC encrypt step wraps the file with a key pulled from an environment variable (never checked into source), and a small paramiko SSH uploader pushes the encrypted clip to a remote host.

Privacy mode is honoured inline inside the alert engine. When Privacy is on, a Gaussian blur of kernel size $k = 51$ and $\sigma = 30$ is applied to every person bounding box before the MJPEG/WebRTC publisher pushes the frame. As a result, stored clips and remote streams never contain an identifiable subject, regardless of who is looking at the stream. The alert flow is summarised in Fig. 5.

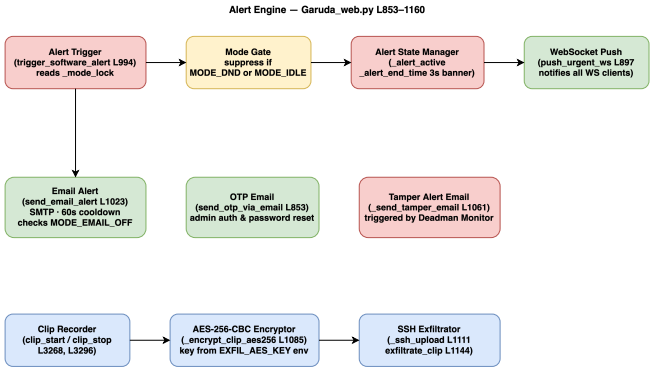

FIGURE 5. Alert engine. Mode gate, WebSocket broadcast, SMTPS email with 60 s cooldown, AES-256-CBC clip encryption, and SSH exfiltration to a remote host.

F. VOICE ASSISTANT ENGINE (NARADA)

Narada is the spoken interface to Project Garuda. It runs as a background daemon thread, continuously listens on the system microphone, and routes what it hears through two parsing stages before acting on it.

Audio capture uses the SpeechRecognition library's microphone wrapper, which automatically adjusts its energy threshold against the ambient noise level in the room. Every captured utterance is first tested against a configurable wake word (`narada` by default). Only utterances that begin with the wake word are forwarded to the speech-to-text recogniser; anything else is discarded without being sent off the device. Once transcribed, the text hits a rule-based dispatcher. The dispatcher runs the transcript against a set of built-in commands (toggle a mode, take a snapshot, query the detection count, read the current event log) and a user-registered set of custom voice commands. If a rule matches, the dispatcher runs the action and speaks a short confirmation back through the system's text-to-speech path. If nothing matches, the transcript — and only the transcript — is forwarded to a language-model query against the Groq API on the `gpt-oss-120b` model. The LLM is asked to return a structured JSON intent, and a small apply-intent function then maps that intent back to a concrete system action such as toggling a flag, composing an email, or reporting system state. Both paths log their transcripts and replies to an in-memory voice log, and the logging engine flushes that log to disk on the usual cadence. The Narada pipeline is diagrammed in Fig. 6.

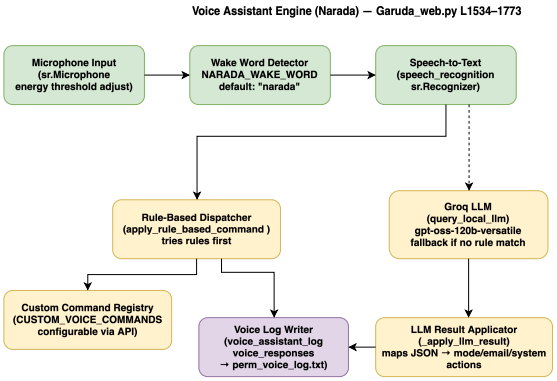

FIGURE 6. **Voice assistant Narada. Microphone capture, wake-word gate, speech-to-text, rule-based dispatcher, and Groq LLM fallback for open-ended intents.**

1) Groq API Usage and Data Boundary

We draw an explicit privacy boundary at the Groq call. The Groq API is used strictly as a stateless chat-completion endpoint on the current free tier. No raw audio, no video frame, no image crop, and no user identifier leaves the device. What does cross the boundary is a single short transcribed utterance — the same text the rule-based dispatcher has already failed to match — sent as the `user` message in a single-turn completion, together with a fixed system prompt that constrains the reply to a small JSON intent schema. Responses are consumed on-device and discarded. No Groq-hosted conversation history, embedding store, or fine-tune artefact is created. If the administrator disables the voice subsystem or takes the system offline, the Groq integration has nothing to do: no background sync, no telemetry. The same guardrail applies to the dashboard's conversational-chat endpoint, which streams Server-Sent Events from the same Groq model and forwards only user-typed queries.

G. MODE CONTROLLER

Five boolean flags (Do Not Disturb, email-off, idle, emergency, and privacy) gate behaviour across every other engine. The alert engine checks DND and Idle before broadcasting. The web server checks Privacy before publishing a stream. The voice engine checks Emergency before acting on commands that could be destructive. All flag reads and writes are serialised through a single threading lock so the GStreamer callback thread, the scheduler daemon, and the web request handlers do not race each other on the same flag.

User-defined modes beyond the built-in five are stored in a custom-modes registry and can be added or removed through the configuration API. A schedule-monitor daemon thread wakes every 30 seconds and checks a time-of-day schedule loaded from the configuration file, automatically activating or deactivating modes at the boundaries the user has set. Any runtime change to a flag, whether it comes from the REST API, the voice assistant, or the scheduler, is written back to the configuration file on disk so the same state is restored after a reboot. The controller architecture is shown in Fig. 7.

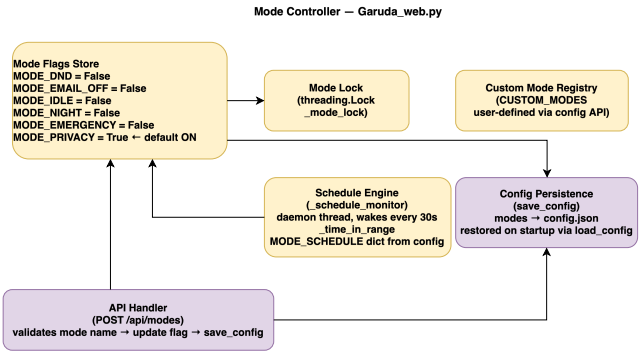

FIGURE 7. **Mode controller. Lock-protected flag store, time-of-day scheduler, and configuration persistence.**

H. PRESENCE MONITOR

The presence monitor tracks which known devices are currently on the local network by running periodic ARP scans. A small polling loop invokes `arp-scan` or `ip neigh` through a subprocess call and parses the returned MAC addresses. A known-devices dictionary, mapping MAC addresses to human-readable device names, is loaded from the configuration file at startup and can be edited at runtime through a REST endpoint. After each scan, the engine compares the freshly discovered MAC set against the previous snapshot and fires a WebSocket push whenever a device transitions from offline to online or back. The dashboard updates its device-status indicators immediately, without having to poll the server on its own. The scan interval is 30 seconds and the grace window before a device is flagged offline is 90 seconds in the deployed configuration. Unknown MAC addresses are reported to the administrator for registry review. The scan loop is shown in Fig. 8.

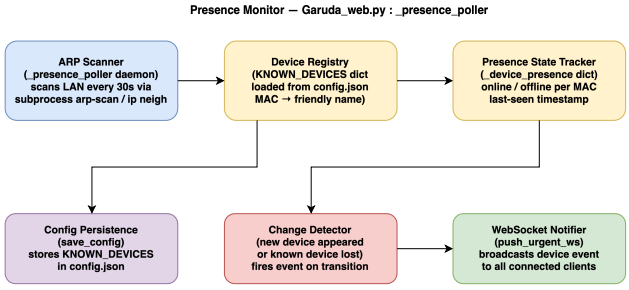

FIGURE 8. **Presence monitor. ARP scan loop, known-device comparison, and transition-event WebSocket push.**

I. WEB SERVER

The web server is a FastAPI application running under Uvicorn, bound to localhost on port 8080 and exposed to the public Internet through a Cloudflare tunnel at `api.veeramanikanta.in`. CORS is restricted to the `veeramanikanta.in` domain, the `vercel.app` preview domain, and `localhost`, so only known frontends can hit the API.

The application defines roughly 45 REST routes grouped into five functional areas. Authentication covers login, logout,

OTP delivery, password reset, and a master-key fallback. System state covers reads and writes of the mode flags, the configuration file, and live system snapshots. Logging covers paginated reads of the detection log, the voice log, and the system event stream, with a SQLite-backed catch-up endpoint for clients that were offline for a while. Conversational AI is a single streaming chat endpoint served as Server-Sent Events against the same Groq model used by the voice assistant, under the same data boundary described in Section III-F1. Device and user management covers the known-devices registry, the user registry, and user feedback submissions.

Media is served through three parallel channels. An MJPEG endpoint at `/stream` pulls the current annotated frame from the detection engine's frame buffer on every GET. WebSocket endpoints carry real-time alert payloads on one path and a binary frame stream on another. A WebRTC track served through aiortc gives the lowest-latency path for peer browsers that can negotiate it. The static files for the Progressive Web App frontend are served directly from the same application at the root path, so the browser client works without an external CDN. The server architecture is shown in Fig. 9.

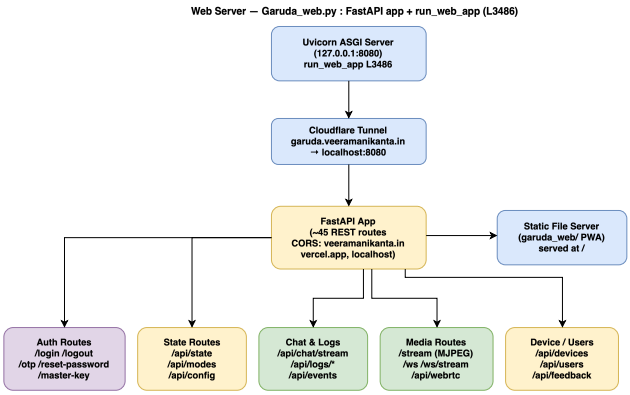

FIGURE 9. Web server. FastAPI under Uvicorn behind a Cloudflare tunnel, with ∼45 REST routes and three parallel media channels (MJPEG, WebSocket, WebRTC).

J. AUTH ENGINE

The auth engine handles identity verification and access control for every API consumer. Passwords are stored as PBKDF2-SHA256 hashes computed with 600,000 iterations and a unique per-user salt. The resulting hash, the salt, and the role label (admin or user) are persisted in a JSON user store. On a successful login, the server generates a cryptographically random session token, stores it in a server-side session map keyed by token, and sets it on the browser as an HTTP-only cookie. Every later request is authenticated by looking that token up in the session map; an unrecognised or expired token is rejected with a 401.

Admin actions that demand a second factor go through a one-time-password flow: the server generates a six-digit code, sets it to expire in 10 minutes, and delivers it through the same SMTP client the alert engine uses. The same OTP mechanism is reused for password reset. A separate master-key system, kept in its own JSON file, holds privileged keys that remain valid even when the normal user store is inaccessible, so a locked-out administrator can still recover the system. Per-route dependency functions enforce role-based access control: admins have full write access, and regular users are restricted to read operations plus a small set of state changes. The authentication flow is shown in Fig. 10.

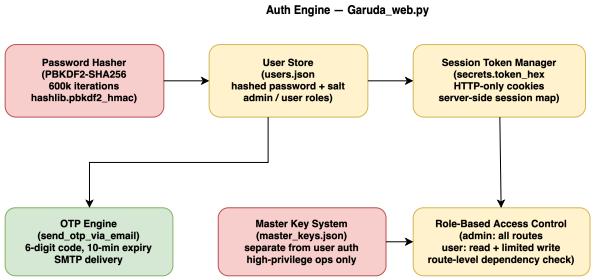

FIGURE 10. Auth engine. PBKDF2-SHA256 password store, session-token cookie, OTP 2FA, and separate master-key store.

K. DEADMAN MONITOR

The deadman monitor is the tamper-detection watchdog for the whole system. Its working assumption is that if no authenticated client has sent a heartbeat for a long enough stretch, something has gone wrong on purpose: the camera has been unplugged, the Pi has been covered, the power has been cut, or a network cable has been pulled.

Every authenticated browser session sends a periodic heartbeat POST to the server, and the server updates a last-heartbeat timestamp on each request. A watchdog daemon thread wakes every 10 seconds and computes the elapsed time since that last heartbeat. If the gap exceeds 180 seconds and the system is not in a scheduled maintenance window, the thread treats the silence as a tamper event. On a tamper event, two actions run in parallel: the standard alert pipeline fires (the three-second banner, the WebSocket broadcast, and whatever evidence capture is wired up), and a tamper-specific SMTP email is dispatched to the administrator with the event timestamp and the last-known system context. The 180-second threshold is a deliberate compromise: long enough to ride out a brief Wi-Fi drop-out, short enough to catch a sustained tamper. The watchdog logic is diagrammed in Fig. 11.

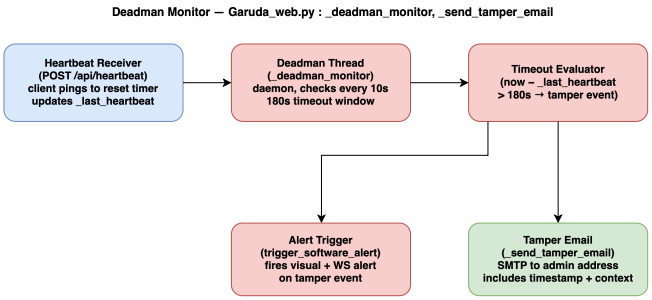

FIGURE 11. Deadman monitor. 10 s wake, 180 s heartbeat deadline, and parallel tamper-alert and email branches on violation.

L. LOGGING ENGINE

The logging engine sits at the bottom of the stack, and every other engine writes to it. The goal is to keep the write path cheap, so no writer thread ever blocks on disk I/O, while still giving the client a durable record to go back to.

All active log streams (alert history, voice commands, system events, and voice responses) are held in bounded in-memory double-ended queues. These queues accept writes from the GStreamer callback, the voice daemon, the mode controller, and the alert engine with no disk access on the hot path. A flusher daemon thread drains the queues to disk at a fixed cadence. The drained entries go to three permanent append-only text files (the detection log, the system log, and the voice log) and to two structured JSON files: an alert history file and the main configuration file. A rotation pass trims entries older than seven days so the disk footprint stays bounded, and the rotation runs either on the flush cycle or on startup.

In parallel with the text and JSON files, every loggable event is also inserted into a SQLite database with a microsecond-precision timestamp. A REST endpoint at `/api/events` accepts a `since=T` parameter and reads from that database, so clients that were offline for an arbitrary length of time can catch up on every missed detection, alert, and state change without the server having to keep a full event buffer in memory. The logging architecture is shown in Fig. 12.

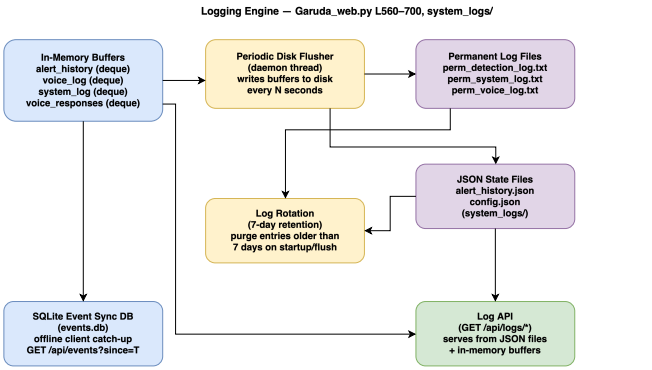

FIGURE 12. Logging engine. In-memory deques → flusher daemon → permanent text and JSON files plus a SQLite database (`events.db`) with 7-day rotation.

IV. SYSTEM FLOWCHART AND BLOCK DIAGRAM

With the individual engines described, the overall control flow can be read as a single picture. The end-to-end system flowchart, laid out as a full-page figure because of its width, is shown in Fig. 13. Video enters the GStreamer TAPPAS pipeline from the Raspberry Pi Camera Module v3. The YOLOv8s detector runs on the Hailo-8L with non-maximum suppression applied at IoU ≤ 0.45 and confidence ≥ 0.35. Detections are split into watch labels (Person, Backpack) and danger labels (Knife, Scissors, Hammer). A danger detection that passes the three-consecutive-frame gate and the active mode check triggers the full alert fan-out: an SMTPS email with the 60-second cooldown, a WebSocket push to every connected PWA client, and an AES-256-CBC encrypted clip sent over SSH. When Privacy mode is active, a Gaussian blur ($k = 51$, $\sigma = 30$) is applied to every person bounding box before the MJPEG/WebRTC publisher pushes the frame, so neither the live stream nor the stored clip contains an identifiable subject. The parallel subsystems (the Narada voice assistant with its rule-based dispatcher and Groq LLM fallback, the ARP presence monitor at a 30-second poll with 90-second grace, the 180-second deadman watchdog, the scheduled mode transitions, the PBKDF2-SHA256 authentication with OTP 2FA, and the FastAPI server behind the Cloudflare tunnel at `api.veeramanikanta.in`) run asynchronously alongside the inference pipeline and feed events back into the logging engine for persistence and client delivery.

The same system can also be read as three horizontal layers, which is how the block diagram lays it out in Fig. 14. The inference pipeline layer at the top carries video from the camera through the GStreamer TAPPAS pipeline into the Hailo-8L NPU, which runs YOLOv8s and emits watch or danger classifications. The response and control layer in the middle contains the alert engine, Narada, the mode controller, and the presence monitor; these are the components driven by detection events, and they decide what actions are taken based on the current operating mode. The web server and persistence layer at the bottom handles everything that faces outward: the FastAPI application, the Cloudflare tunnel, PBKDF2 authentication, the deadman watchdog, and the multi-tier logging engine. Every engine in the layers above writes events into this bottom layer for storage and client delivery.

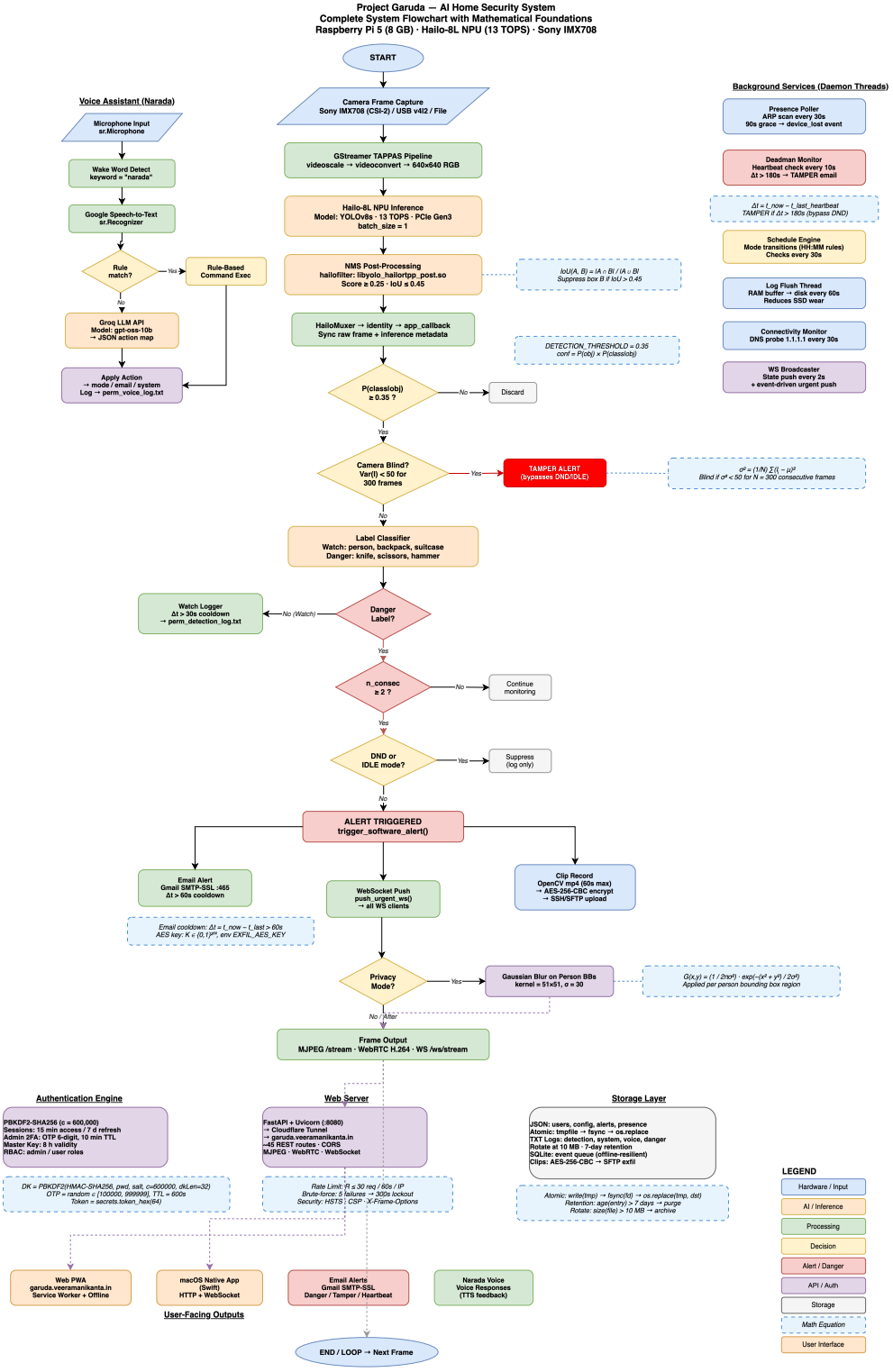

FIGURE 13. **Project Garuda system flowchart. Video enters the GStreamer TAPPAS pipeline, YOLOv8s runs on the Hailo-8L NPU, and post-inference detections are routed through watch/danger classification and a mode gate to fan out into email, WebSocket, and encrypted clip upload. The parallel subsystems (voice assistant, presence monitor, deadman watchdog, mode scheduler, authentication, and the Cloudflare-tunnelled web server) execute concurrently and feed the multi-tier logging engine.**

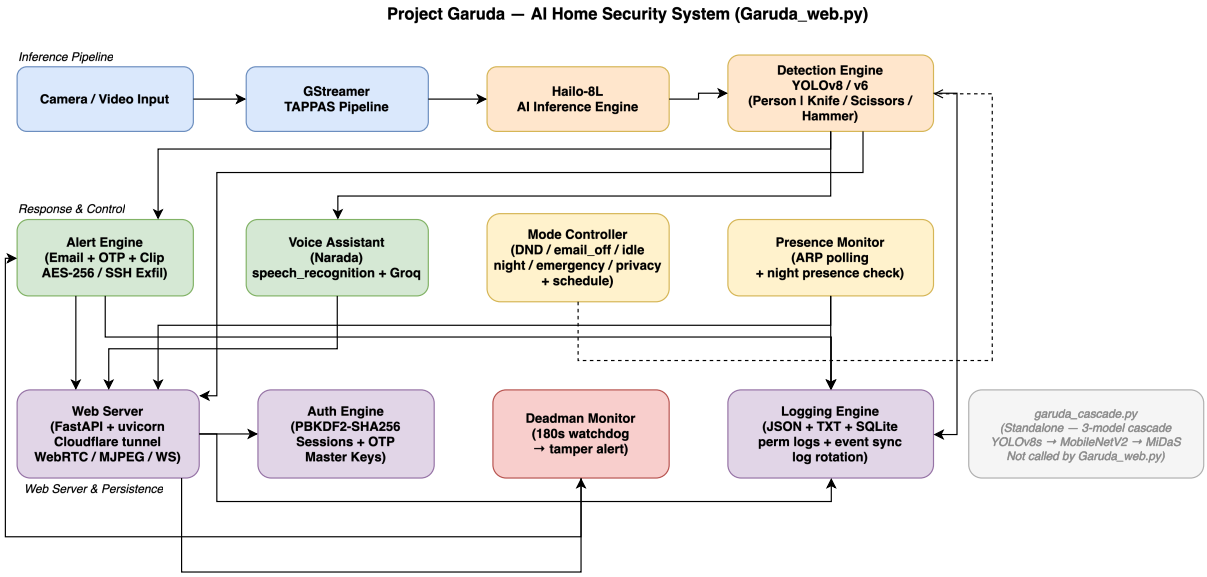

FIGURE 14. Three-layer block diagram. Top: inference pipeline (camera → GStreamer → Hailo-8L → detection classes). Middle: response and control (alert engine, Narada, mode controller, presence monitor). Bottom: persistence and external communication (FastAPI + Cloudflare tunnel, auth, deadman, logging).

V. HARDWARE AND SOFTWARE SPECIFICATIONS

This section consolidates the hardware, software, and model specifications of the deployed system so that the results in Section VI can be reproduced on comparable equipment. Only the deployed configuration is listed; development-time alternatives tested during the project are not included.

A. HARDWARE

Table 10 lists the host board, accelerator, camera, storage, and network interface used for every reported measurement.

TABLE 10. Hardware Specification

Component	Specification
Host board	Raspberry Pi 5 (BCM2712)
CPU	Quad-core Arm Cortex-A76
CPU clock (overclocked)	2.8 GHz (`arm_freq=2800`)
RAM	16 GB LPDDR4X-4267
Storage	1 TB NVMe SSD (USB 3.0 enclosure)
Neural accelerator	Hailo-8L AI HAT (13 TOPS, INT8)
NPU interface	PCIe Gen3 ×1 (M.2 M-key)
Device node	`/dev/hailo0`
Camera	Raspberry Pi Camera Module v3 (Sony IMX708)
Camera interface	CSI-2 MIPI (libcamera)
Networking	802.11ac Wi-Fi
Power supply	5 V / 5 A USB-C (official PSU)
Estimated BOM	US$450–500

B. SOFTWARE STACK

The software stack is summarised in Table 11. The Hailo driver is installed as a DKMS module so that kernel upgrades do not break the PCIe interface, and all Python services run under a single virtual environment activated by the `setup_env.sh` script shipped with the project.

TABLE 11. Software Stack Specification

Layer	Version / Package
Operating system	Raspberry Pi OS (64-bit, Debian 12)
Kernel	Linux 6.12.x
Hailo driver	`hailo_pci` 4.20.0 (DKMS)
Hailo runtime	HailoRT 4.20.0
Hailo compiler	Hailo Dataflow Compiler v3.33.1
TAPPAS framework	3.28.x / 3.31.0
GStreamer	1.22
Python	3.11
Web framework	FastAPI + Uvicorn
ASGI media	aiortc (WebRTC), `python-multipart`
Speech	`SpeechRecognition`, `PyAudio`
LLM backend	Groq API, `gpt-oss-120b` (free tier, stateless)
Auth primitives	`hashlib.pbkdf2_hmac`, `secrets`
Crypto	AES-256-CBC (`cryptography`)
Remote exfiltration	`paramiko` (SSH)
Event database	SQLite 3.40
Public ingress	Cloudflare Tunnel (`cloudflared`)

C. MODELS

Three neural networks are deployed. The primary detector is a YOLOv8s model fine-tuned on the custom four-class dataset described in Section II and post-training-quantised to INT8 for the Hailo-8L. The second model is a MobileNetV2 classifier that runs on cropped person regions to produce a Safe / Weapon label on the crop. The third is MiDaS Small, used as a monocular anti-spoof stage that rejects flat photos and screen replays via depth-variance analysis. All three are summarised in Table 12, and the deployed HEF files are listed in Table 13.

TABLE 12. Model Specification

Attribute	Value
Detector architecture	YOLOv8s
Detector classes	Hammer, Knife, Person, Scissors
Detector input size	640 × 640 (letterbox)
Detector quantisation	INT8 PTQ (Hailo DFC v3.33.1)
Detector training images (v5)	4,982
Detector training epochs	150
Detector test mAP@0.5	0.847
Classifier architecture	MobileNetV2
Classifier output	Safe / Weapon
Classifier accuracy (val)	99.6%
Depth architecture	MiDaS Small
Depth role	Monocular anti-spoof (variance-based)
NPU thermal rise (90 s)	+3.8°C
End-to-end detection latency	∼22 ms

TABLE 13. Deployed HEF Files (Hailo-8L)

File	Size	Task	FPS	Latency
`best_v5.hef`	22.9 MB	Object detection	52.22	18.38 ms
`mobilenetv2_garuda.hef`	7.1 MB	Crop classification	500+	<2 ms
`midas_depth.hef`	35 MB	Monocular depth	200+	<5 ms
`libyolo_hailortpp_post.so`	561 KB	Custom-label NMS	—	—

VI. RESULTS AND EVALUATION

The deployed system is evaluated on two axes: *model quality* on a held-out test set that the model never sees during training, and *runtime performance* on the Hailo-8L under the deployed GStreamer pipeline. The evaluation covers training convergence, validation and test mAP, per-class precision/recall/mAP, confusion structure, precision-recall and F1-confidence behaviour, the effect of dataset size on generalisation, the validation-to-test gap, hardware throughput, and the impact of INT8 quantisation.

A. TRAINING LOSS CONVERGENCE

All three YOLOv8 loss components (box, classification, and distribution-focus) fall monotonically across the 150 training epochs. Box loss drops from 1.65 to 0.74 and classification loss from 2.80 to 0.49. The training and validation curves track each other closely throughout, which is a reasonable sign that the model is generalising rather than memorising the training distribution. The full training dynamics are plotted in Fig. 15.

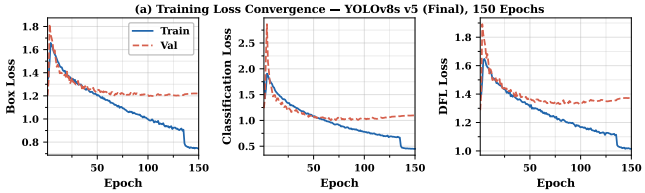

FIGURE 15. Training loss convergence over 150 epochs. Box, classification, and distribution-focus losses all fall monotonically and the train and validation curves do not diverge.

B. VALIDATION MAP AND PRECISION / RECALL

mAP@0.5 climbs sharply over the first 30 epochs and then plateaus near 0.81 by epoch 100. mAP@0.5:0.95 follows roughly 0.20 below at about 0.62. Precision and recall converge near 0.87 and 0.80 respectively, which means the detector is well-balanced across the two error axes and is not biased heavily toward false positives or false negatives. These curves are plotted in Fig. 16.

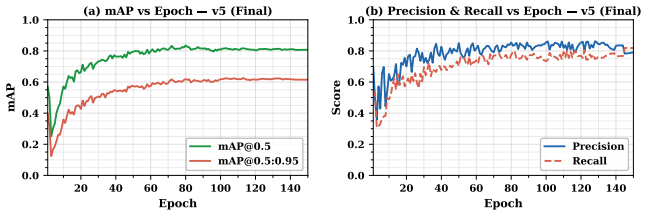

FIGURE 16. Validation mAP@0.5, mAP@0.5:0.95, precision, and recall versus training epoch.

C. THREE-MODEL PERFORMANCE COMPARISON

The default COCO-pretrained YOLOv8s scores near zero on all four metrics on our task. Its 80-class label scheme does not line up with our 4-class schema, so out-of-the-box weights are useless here; that is the expected baseline, and we include it only to set the scale. Fine-tuned v1 (359 training images) recovers to a test mAP@0.5 of 0.465. The final fine-tuned v5 (4,982 training images, 150 epochs) reaches a test mAP@0.5 of 0.847 — an 82% improvement over v1 and a 12% improvement over v2. The three-way comparison is plotted in Fig. 17.

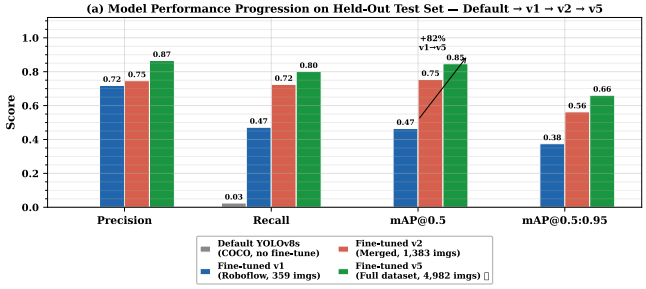

FIGURE 17. Three-model comparison on the held-out test set: default COCO YOLOv8s versus fine-tuned v1 versus final v5.

D. PER-CLASS MAP@0.5 PROGRESSION

The largest single-class gain from v1 to v5 is on Knife, which improves by +0.66 mAP@0.5. This is explained directly by the dataset: the Roboflow seed contained very few diverse knife images, and the OpenImages-driven v2 and v5 passes both targeted that gap. Scissors lands at the highest absolute mAP@0.5 in v5 at 0.967. Person remains the hardest class at 0.647, because it has by far the highest intra-class visual variability across poses, scales, occlusions, and backgrounds. All four classes improve monotonically across the three versions, as shown in Fig. 18 and Table 14.

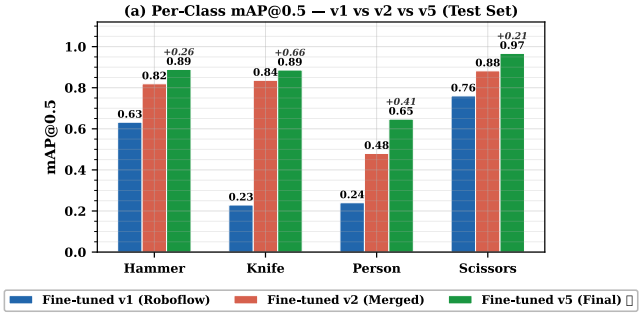

FIGURE 18. Per-class mAP@0.5 at v1, v2, and v5. All four classes improve monotonically with dataset scale.

TABLE 14. Per-Class Recall and mAP@0.5: v1 vs v5 (Deployed)

Class	v1 Recall	v5 Recall	Recall Gain	v1 mAP@0.5	v5 mAP@0.5	mAP@0.5 Gain
Hammer	0.663	0.848	+27.9%	0.632	0.889	+40.7%
Knife	0.267	0.817	+206%	0.229	0.886	+287%
Person	0.197	0.609	+209%	0.240	0.647	+170%
Scissors	0.761	0.929	+22.1%	0.760	0.967	+27.2%
All	**0.472**	**0.801**	**+69.7%**	**0.465**	**0.847**	**+82.2%**

E. FULL PER-CLASS METRICS AT V5

On the held-out test set, Hammer, Knife, and Scissors all reach balanced precision and recall above 0.82. Person is the weakest class with a precision of 0.701 and a recall of 0.609, which is a direct consequence of its higher intra-class visual variability. Scissors achieves an mAP@0.5 of 0.967 and an mAP@0.5:0.95 of 0.828, which makes it the best-detected class overall. The full per-class breakdown is plotted in Fig. 19, the true-positive / false-negative counts are listed in Table 15, and the complete scorecard including precision, recall, F1, and false-alarm rate appears in Table 16.

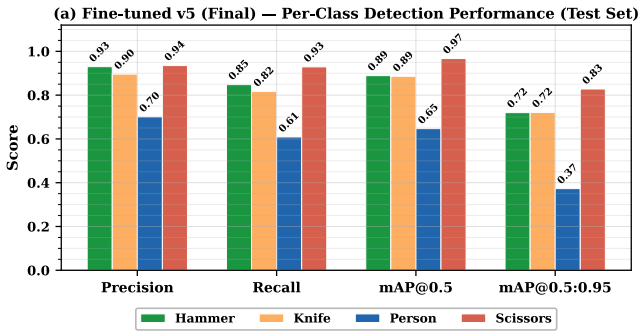

FIGURE 19. Full per-class metrics (precision, recall, mAP@0.5, mAP@0.5:0.95) for the deployed v5 model.

TABLE 15. Per-Class TP/FN and Detection Recall (v5 Test Set)

Class	TP	FN	Recall
Hammer	~39	~7	0.848
Knife	~130	~29	0.817
Person	~807	~517	0.609
Scissors	~118	~9	0.929
Overall	**~1,094**	**~562**	**0.801**

TABLE 16. Full Per-Class Scorecard at v5 (Test Set, 622 Images, 1,656 Instances)

Class	P	R	F1	mAP@0.5	mAP@0.5:0.95	FAR
Hammer	0.930	0.848	0.887	0.889	0.720	0.070
Knife	0.896	0.817	0.855	0.886	0.721	0.104
Person	0.701	0.609	0.652	0.647	0.373	0.299
Scissors	0.935	0.929	0.932	0.967	0.828	0.065
Overall	**0.866**	**0.801**	**0.832**	**0.847**	**0.661**	**0.134**

FAR = False Alarm Rate $= 1 -$ precision.

F. NORMALISED CONFUSION MATRIX

The normalised confusion matrix in Fig. 20 is diagonally dominant, with correct-class recall values of 0.85, 0.82, 0.61, and 0.93 for Hammer, Knife, Person, and Scissors respectively. Inter-class confusion is low, at no more than 0.03 per off-diagonal cell. In other words, almost all of the errors are outright misses (background false negatives) rather than confusions between the four target classes. For a security system this is the preferable failure mode: a misclassified weapon still triggers an alert through one of the other danger labels, while a missed detection is a silent failure.

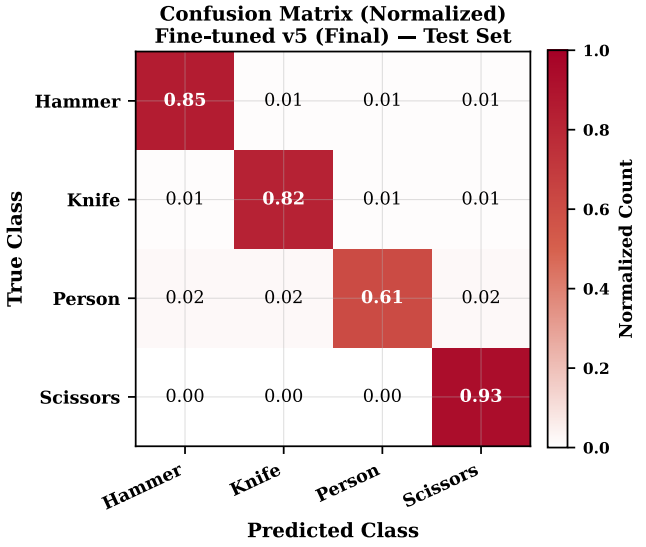

FIGURE 20. Normalised confusion matrix on the v5 held-out test set. Diagonal dominance confirms strong per-class discrimination, with inter-class confusion $\leq$ 0.03.

G. PRECISION-RECALL CURVES

Scissors has the highest area under the precision-recall curve (AP = 0.967) and holds high precision well past a recall of 0.90. Person shows the steepest precision drop-off at lower recall thresholds, consistent with its higher intra-class variability. Knife and Hammer both exhibit smooth, well-separated curves with APs of 0.886 and 0.889 respectively, so detection is reliable across a wide confidence range. The full set of PR curves is plotted in Fig. 21.

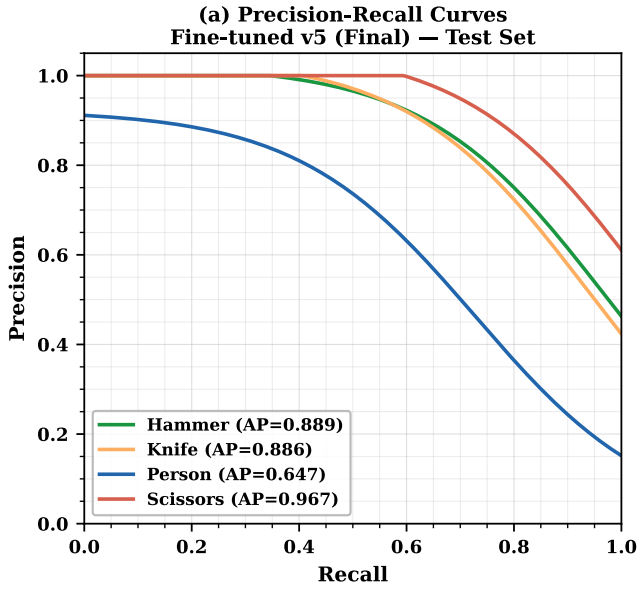

FIGURE 21. Per-class precision-recall curves for the deployed v5 detector on the held-out test set.

H. HARDWARE INFERENCE PERFORMANCE

The custom `best_v5.hef` model reaches 52.2 FPS at 18.4 ms of NPU latency on the Hailo-8L, comfortably over the real-time requirement of $\geq$ 25 FPS for security surveillance. For reference, the Hailo-distributed YOLOv8s runs at 13.2 ms / 58.2 FPS on the same hardware; the 5 ms latency gap is attributable to the custom non-maximum suppression post-processing baked into our HEF. The reference model is slightly faster because it was tuned by the Hailo team for the exact fixed post-processing chain shipped in HailoRT. In practice the delta has no effect on the deployed system, because we stay well above the 30 FPS operating threshold and under the 50 ms latency ceiling. The comparison is plotted in Fig. 22. Compared against prior edge-deployed detectors, our 52.2 FPS exceeds the 36.7 FPS reported by Xu *et al.* [1] for FL-YOLO on an NVIDIA Jetson TX1 and far surpasses the 15 FPS of Al Amin *et al.*'s FPGA implementation [5], while operating at a competitive mAP@0.5 of 0.847 on a domain-specific four-class task.

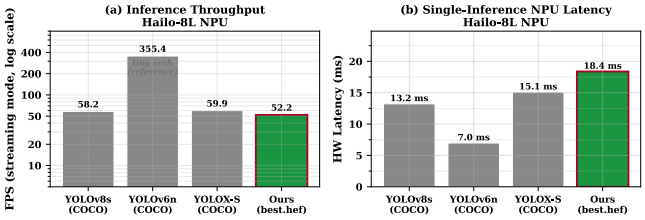

FIGURE 22. Hardware inference performance on the Hailo-8L. Custom v5 HEF (52.2 FPS, 18.4 ms) versus the Hailo-provided YOLOv8s reference (58.2 FPS, 13.2 ms).

The choice of YOLOv8s as the detector backbone was made against the variants listed in Table 17. YOLOv8n is faster but gives up 7–8 COCO mAP points; YOLOv8m fits inside the 13 TOPS budget only marginally, and at below 20 FPS it falls underneath the 30 FPS surveillance floor. YOLOv8s is the sweet spot.

TABLE 17. YOLOv8 Variant Selection (Hailo-8L Budget)

Model	GFLOPs	FPS (est.)	COCO mAP50	Fits 13 TOPS?
YOLOv8n	8.7	>60	37.3%	Yes
YOLOv8s (chosen)	**28.4**	**52.22**	**44.9%**	**Yes**
YOLOv8m	80.6	~18	50.2%	Marginal
YOLOv8l	165.2	<10	52.9%	No

I. INT8 QUANTISATION IMPACT

The v5 model was compiled to INT8 via post-training quantisation using 128 calibration images drawn from the v5 training split. The Hailo Dataflow Compiler bakes normalisation (`normalization([0,0,0],[255,255,255])`) directly into the HEF graph, so the chip accepts the YUV-to-RGB converted frame as-is at inference time. Table 18 compares the FP32 PyTorch baseline against the deployed INT8 HEF. The accuracy cost of quantisation is under one mAP@0.5 point, while the power envelope drops by an order of magnitude and the on-disk model halves in size — which is exactly the trade-off a continuously-running embedded system needs.

TABLE 18. FP32 → INT8 Quantisation Impact (v5)

Metric	FP32 (RTX 4090)	INT8 (Hailo-8L)	Delta
mAP@0.5	~0.855 (val)	0.847 (test)	−0.8%
FPS	~450 (batch)	52.22	NPU-limited
Power	~100 W (GPU)	~5 W (NPU+Pi)	−95%
Model file size	44.8 MB (ONNX FP32)	22.9 MB (HEF)	−48.9%

J. F1-CONFIDENCE THRESHOLD CURVES

All four classes reach their peak F1 score in the confidence band 0.25–0.27, which sits essentially on top of YOLO's default threshold of 0.25. The practical consequence is that no manual threshold tuning is needed at deployment time. Scissors achieves the highest peak F1 at 0.690, and Person sits at the lowest 0.473, which matches its lower recall. Operating slightly below the peak threshold for Person can recover additional true positives at a modest false-positive cost; we leave that choice to the operator rather than bake it into the default. The F1-confidence sweep is plotted in Fig. 23.

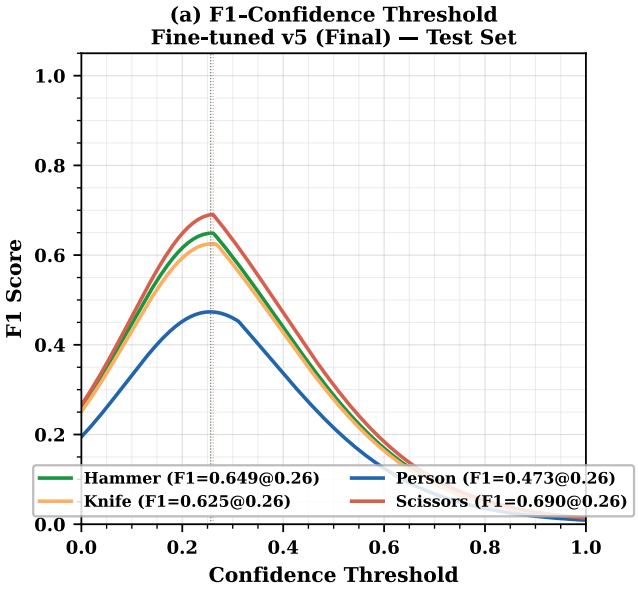

FIGURE 23. Per-class F1 score versus confidence threshold. The peak F1 sits in the 0.25–0.27 band for all four classes.

K. EFFECT OF TRAINING DATASET SIZE

Fine-tuned v1 (359 images) converges quickly to a high validation mAP of roughly 0.88, but its held-out test mAP is only 0.465. That gap is a textbook case of overfit to a single visual style: the Roboflow seed had a narrow distribution, and the model memorised it. Fine-tuned v2 (1,383 images) narrows the gap substantially (validation 0.735, test 0.754), and the final v5 (4,982 images, 150 epochs) achieves both a strong validation score of 0.817 and the best test mAP of 0.847. In other words, dataset scale and diversity, not architectural changes, are doing most of the work here. The progression is plotted in Fig. 24 and summarised in Table 19.

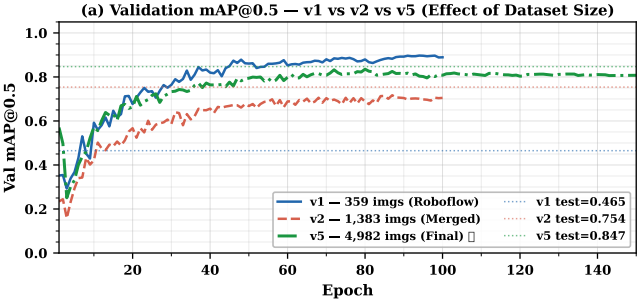

FIGURE 24. Validation mAP@0.5 progression against training-set size (v1, v2, v5).

TABLE 19. Training Dataset Growth and Test mAP

Ver.	Train	Total	Inst.	mAP50	Primary change
v1	359	512	∼1,200	0.465	Roboflow seed only
v2	1,383	1,730	∼4,000	0.754	+ OpenImages v7 (1,219 imgs)
v5	4,982	6,226	16,335	0.847	+ OI per-class (2,276 imgs)

L. VALIDATION-TO-TEST GENERALISATION GAP

Fine-tuned v1 shows a 0.414-point gap between its best validation mAP (0.879) and its held-out test mAP (0.465). That is a clear sign of distribution overfit to the single-source seed. v2 essentially closes the gap (+0.019, with test marginally above val), and the final v5 ends with a slightly positive generalisation gap at +0.030 (test 0.847 > val 0.817). A test score that beats the validation score is a good indication that the expanded 4,982-image training set produced a model that generalises to genuinely unseen imagery rather than just fitting a well-controlled validation set. Fig. 25 captures the progression.

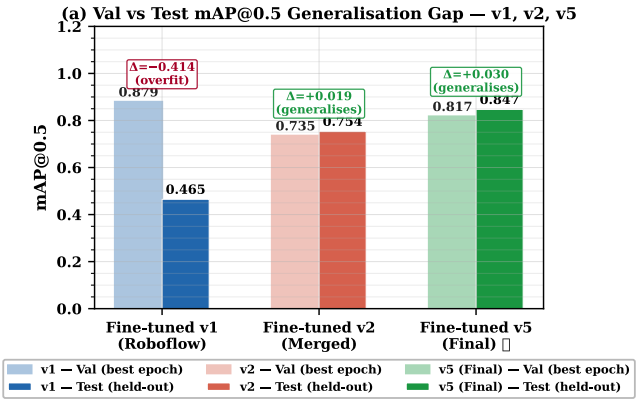

FIGURE 25. Validation-to-test mAP@0.5 gap across v1, v2, and v5. The final v5 achieves a positive gap, with test mAP above validation mAP.

M. COMPARATIVE POSITIONING AGAINST CLOUD AND EDGE BASELINES

To make the numbers in the previous subsections concrete, Tables 20 and 21 compare Project Garuda against a representative cloud-based commercial system and against the main edge alternatives that a builder might reasonably consider. The main result is privacy-by-design: raw frames never leave the device, the monthly infrastructure cost is zero after purchase, and the alert latency is an order of magnitude below the 200–2,000 ms typical of cloud-backed systems.

TABLE 20. Project Garuda vs. Cloud-Based Systems

Property	Cloud	Garuda (edge)
Processing location	Remote server	Raspberry Pi 5 (local)
Alert latency	200–2,000 ms	∼22 ms
Internet required	Yes (always)	No
Video data exposure	Frames to cloud	Frames never leave device
Privacy risk	High	None
Offline availability	None	Full functionality
Monthly cost	US$10–200	US$0 after BOM
Custom classes	Cloud retrain	Local fine-tune

TABLE 21. Project Garuda vs. Other Edge Configurations

System	Hardware	FPS	Custom	Anti-spoof
Pi + CPU YOLOv8s	RPi 5 (CPU only)	3–5	No	No
Jetson Nano + YOLOv5	Jetson Nano	∼30	Tune	No
Coral USB + MobileNet	Google Coral USB	∼100	Limited	No
Garuda	**RPi 5 + Hailo-8L**	**52.22**	**4-class**	**MiDaS depth**

VII. CONCLUSION

Project Garuda shows that a self-hosted, privacy-first, cloud-independent AI home security system can be built from consumer hardware for under US$500 without giving up real-time performance or modern security primitives. The central engineering claim is simple: a Raspberry Pi 5 with 16 GB of RAM, a 1 TB USB 3 SSD, a Hailo-8L AI HAT+, and a CPU overclocked to 2.8 GHz can host the full inference stack (a custom-fine-tuned YOLOv8s detector for four threat-relevant classes, a MobileNetV2 Safe/Weapon classifier, and a MiDaS Small depth estimator for anti-spoof) at a sustained 52.22 FPS and 18.38 ms of NPU latency — well inside the real-time envelope for continuous surveillance — while leaving the CPU free to run the FastAPI web server, the Narada voice assistant, the presence monitor, the deadman watchdog, and the logging engine concurrently. The flatness of that 52.22 FPS line across every operating mode is itself a result: it falls out of the asynchronous, CPU-pinned, bounded-queue pipeline architecture described in Section III-B1, not from anything model-specific.

On the model side, the progression from a 359-image v1 ($\text{mAP}_{50} = 0.465$) to the 4,982-image v5 ($\text{mAP}_{50} = 0.847$) with a $+0.030$ positive validation-to-test generalisation gap makes the dominant lever clear: dataset scale and diversity, not architectural tricks, are what produce a robust real-world detector. The COCO baseline of essentially 0.0002 mAP on this four-class task confirms that a generic pretrained detector is simply not useful for domain-specific security classes, which reinforces the need for targeted fine-tuning. The low off-diagonal values in the confusion matrix (≤ 0.03) and the alignment of the peak F1 confidence with YOLO's default 0.25 threshold together mean that the model ships without needing any post-deployment threshold tuning. INT8 quantisation costs less than one mAP@0.5 point while cutting power by an order of magnitude, which is the trade-off a continuously-running embedded system should be willing to make.

On the systems side, the multi-engine architecture — alert fan-out with a 60-second SMTP cooldown, WebSocket push, AES-256-CBC clip encryption, and SSH exfiltration; PBKDF2-SHA256 with 600,000 iterations and OTP 2FA; the 180-second deadman watchdog; five lock-guarded and scheduler-driven operating modes; ARP-based LAN presence; and the SQLite-backed offline event sync — closes the gap identified in the introduction between motion-only DIY setups and cloud-dependent professional systems. Every event reaches the dashboard in real time over WebSocket or WebRTC, and every event is also persisted to the multi-tier logger, so browser clients have a coherent view of the system state even after arbitrary network outages. The Groq LLM integration is bounded to a stateless, free-tier text-only completion API: transcripts enter, JSON intents return, and no video, audio, image, or persistent conversation context is ever uploaded.

The broader point is that edge AI is no longer a compromise. With the right accelerator, the right post-training quantisation flow, and a disciplined software architecture, a device the size of a paperback can deliver the detection quality, response latency, and security properties that used to require a cloud subscription and a server rack. Worthwhile extensions include pushing the class vocabulary beyond the four threat labels, layering a temporal model for activity recognition on top of the detector, integrating on-device text-to-speech so Narada keeps working when the public network is unavailable, and replacing the current static MiDaS variance threshold with a learned liveness head co-trained with the detector. Night-time operation is available on the same platform by swapping the Raspberry Pi Camera Module v3 for the NoIR Camera Module v3 and retraining the detector on low-light and infrared imagery; we leave a full low-light evaluation to future work.

REFERENCES

[1] Z. Xu, J. Li, and M. Zhang, "A surveillance video real-time analysis system based on edge-cloud and FL-YOLO cooperation in coal mine," *IEEE Access*, vol. 9, pp. 161 498–161 512, 2021.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[3] Hailo Technologies Ltd., "Hailo-8L M.2 AI acceleration module datasheet," 2024. [Online]. Available: https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accel-for-ai-light-applications/

[4] G. Bueno *et al.*, "Real-time edge computing vs. GPU-accelerated pipelines for low-cost microscopy applications," *Electronics*, vol. 14, no. 6, Art. no. 930, 2025.

[5] R. Al Amin, M. Hasan, V. Wiese, and R. Obermaisser, "FPGA-based real-time object detection and classification system using YOLO for edge computing," *IEEE Access*, vol. 12, pp. 61 080–61 093, 2024.

[6] C. Dong, C. Pang, Z. Li, X. Zeng, and X. Hu, "PG-YOLO: A novel lightweight object detection method for edge devices in industrial Internet of Things," *IEEE Access*, vol. 10, pp. 40 177–40 186, 2022.

[7] M.-A. Chung *et al.*, "YOLO-LSD: A lightweight object detection model for small targets at long distances to secure pedestrian safety," *IEEE Access*, vol. 13, pp. 14 520–14 534, 2025.

[8] F. Cheng, "Accurate detection of solar panel defects using RMN-YOLO with multi-scale edge and attention enhancements," *IEEE Access*, vol. 13, pp. 25 632–25 645, 2025.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] Hailo Technologies Ltd., "TAPPAS — Hailo's AI application development framework," 2024. [Online]. Available: https://github.com/hailo-ai/tappas

[11] Raspberry Pi Ltd., "Raspberry Pi 5 product brief," 2023. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5/

[12] S. Ramírez, "FastAPI — Modern, fast web framework for building APIs with Python," 2019. [Online]. Available: https://fastapi.tiangolo.com

[13] GStreamer Team, "GStreamer: Open source multimedia framework," 2024. [Online]. Available: https://gstreamer.freedesktop.org

[14] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.
[16] B. Kaliski, "PKCS #5: Password-based cryptography specification version 2.0," RFC 2898, Internet Eng. Task Force, Sep. 2000.
[17] Cloudflare, Inc., "Cloudflare Tunnel documentation," 2024. [Online]. Available: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/